

Evaluation Harness

(src/eval.py)

1 Metrics and What They Actually Tell You

Per-particle L2 (per_particle_l2)

$$L2(t) = \sqrt{(1/N) \sum_p \|\hat{x}_p(t) - x_p(t)\|^2}$$

The RMS position error per particle vs. the full-MPM reference at each frame. Our target $\tau = 0.05$ m (5 cm) is derived from [paper10]'s FlagDynamic rollout-50 RMSE of $72.4 \times 10^{-3} \approx 7$ cm on a 1 m normalized cloth. We set τ slightly below their 50-step error since our model is smaller (target $L = 3\text{--}5$ vs. their $L = 15$ message-passing rounds), making $\tau = 5$ cm an achievable but meaningful goal.

Chamfer distance (chamfer)

$$\text{Chamfer} = (1/N) \sum_p \min_q \|\hat{x}_p - x_q\|^2 + (1/N) \sum_q \min_p \|\hat{x}_p - x_q\|^2$$

Unlike L2, Chamfer doesn't require particle-to-particle correspondence — each predicted particle finds its nearest reference particle and vice versa. [paper10] focuses on correspondence-based position RMSE; Chamfer is our additional topology-robustness check for cases where the neural solver causes particles to drift to plausible-but-wrong positions.

Energy drift (energy_drift)

$$\text{EnergyDrift}(t) = |\text{KE}_{\text{pred}}(t) - \text{KE}_{\text{ref}}(t)| / \text{KE}_{\text{ref}}(t)$$

A proxy for whether the solver is gaining or losing energy. High energy drift signals numerical instability. [paper13] demonstrates that bend stiffness can vary over a $10,000\times$ range with $< 5\%$ variance in running time — showing that implicit Euler's stability is independent of stiffness. We use energy drift to verify the same property holds for our neural + fallback combination.

Rollout horizon (rollout_horizon)

$$\text{RolloutHorizon}(\tau) = \max\{t : L2(t') \leq \tau \quad \forall t' \leq t\}$$

"How many seconds can we run the neural solver before it drifts past $\tau = 5$ cm?" This is the direct answer to H1. The concept mirrors [paper10]'s observation that cloth rollouts *decohere* after ~50 steps — they report rollout-50 and rollout-all RMSE separately because the 50-step horizon is the more meaningful quality gate. Our horizon metric formalizes this: we compute the exact step where error first exceeds threshold.

Cosine window signal (`cosine_window`) [paper7, §3.2]

$$\text{CosSim}_t = (1/N) \sum_p \cos(\text{mean}(\bar{a}_p[t-2W : t-W]), \text{mean}(\bar{a}_p[t-W : t]))$$

This is *exactly* the D1 detector signal from [paper7]. It compares the mean acceleration direction over two consecutive $W = 10$ -step windows. When the cloth hits something or undergoes rapid deformation, acceleration directions change suddenly — CosSim drops below $rc = 0.8$, flagging the frame as "hard."

[paper7]'s threshold sweep (Table 2) establishes $rc = 0.8$ as the knee of the quality-vs-latency curve:

Threshold rc	Grid RMSE	Time/step (ms)
0.0 (neural only)	0.0232	0.40
0.5	0.0215	0.50
0.7	0.0192	0.61
0.8	0.0169	0.70
0.9	0.0144	0.74

Both the window size $W = 10$ and threshold $rc = 0.8$ are adopted verbatim from [paper7] and locked in `configs/eval.yaml`.

The detector label threshold $\varepsilon = 0.10$ m $= 2\tau$ is a derived choice: it must be larger than τ so the detector has positive samples to learn from. At $\varepsilon = 10$ cm, the cloth is visibly diverging from the MPM reference — above the "noticeable artifact" threshold for a 1 m sheet.

2 Timing Benchmarks

Two functions time the two reference solvers:

`baseline_timing()` → MPM reference: 0.65 ms/step (CPU, 16×16)

implicit_timing() → Implicit solver: 2.89 ms/step (CPU, 16×16)

These are the numbers the hybrid pipeline must beat. [paper7] achieves -11% to -29% end-to-end latency over pure MPM on a 4090 GPU (e.g., Water-Sand 2D: 0.114 s → 0.08 s/frame). On Colab T4, the MPM step is expected to be ~10–50× faster than CPU (compiled Taichi CUDA kernels). The hybrid target is ≥3–5× speedup over the MPM reference.

Note: MPM uses $dt = 1e-4$ s (10,000 substeps/s simulated) while the implicit solver uses $dt = 0.02$ s (50 steps/s simulated). Per simulated second: MPM = $0.65\text{ms} \times 10,000 = 6,500\text{ms}$; Implicit = $2.89\text{ms} \times 50 = 145\text{ms}$. The implicit solver covers simulated time ~45× faster, which is why it is viable as a fallback.

Every timing CSV carries: `git_sha`, `config_hash`, `mean_ms`, `median_ms`, `std_ms`, `p95_ms`, `p99_ms`. The p95/p99 numbers matter for real-time demos where per-frame spikes cause visible stutters. [paper7] reports per-step latency at ~0.7 ms on a 4090 (at $rc = 0.8$); our CPU baseline of 0.65 ms/step gives a reference for the GPU speedup factor.

3 The CLI

Compare predicted vs reference clip

```
python -m src.eval pair \
```

```
--reference data/cloth_trajectories/drape/clip_000000.npz \
```

```
--predicted results/neural_drape_000000.npz \
```

```
--out results/eval_drape.csv
```

MPM baseline timing

```
python -m src.eval baseline-timing \
```

```
--grid 16 16 --grid-resolution 32 --n-steps 200 \
```

```
--out results/timing_mpm_smoke.csv
```

Implicit fallback timing

```
python -m src.eval implicit-timing \
```

```
--grid 16 16 --n-steps 200 \
```

--out results/timing_implicit_smoke.csv

All three subcommands produce CSVs with identical traceability columns.